ISO/IEC JTC1 SC22 WG21 P0929R0
Jens Maurer <Jens.Maurer@gmx.net>
Target audience: EWG, CWG
2018-02-08

P0929R0: Checking for abstract class types

Introduction

Subclause 13.4 [class.abstract] paragraph 3 states:

> An abstract class shall not be used as a parameter type, as a function return type, or as the type of an explicit conversion. Pointers and references to an abstract class can be declared.

This is troublesome, because it requires checking a property only known once a type is complete at a position (declaring a function) where the rest of the language permits incomplete types to appear.

In practice, this introduces spooky "backward-in-time" errors:

```
struct S;
S f();   // #1, ok

// lots of code

struct S { virtual void f() = 0; };  // makes #1 retroactively ill-formed
```

Discussion

Declarations should add information to the state of a program, not retroactively make existing constructs that have been parsed successfully ill-formed.

Beyond the ability to declare functions with incomplete return and parameter types, the standard also allows for the return type to be inspected using decltype(f()) even if it is incomplete. This ability might be used in template metaprogramming, and it should not break if one of the involved types happens to be abstract.

There are two approaches to solving this issue:

- State that a return type or a parameter type is ill-formed if it is complete and an abstract class type at the point of declaration of the function type.
- Never check declarations involving function types whether they involve a return type or parameter of abstract class type. Instead, postpone the check to the definition and the call, where actual objects need to be created, and where the existing text already requires completeness of the types involved.

The first option seems bizarre in that token-by-token identical redeclarations may become ill-formed:

```
struct S;
S f();      // ok, abstract-ness not checked here
struct S { virtual void f() = 0; };
S f();      // error: S is abstract
```

Therefore, the proposed wording chooses the second option.

It is recommended to treat this as a Defect Report against earlier versions of C++.

The suggested feature test macro is `__cpp_abstract_class_checking`.

Wording changes

Change in 8.2.1 [basic.lval] paragraph 9:

> Unless otherwise indicated (~~8.5.1.2 [expr.call]~~ **10.1.7.2 [dcl.type.simple]**), a prvalue shall always have complete type or the void type**; if it has a class type, that class shall not be an abstract class (13.4 [class.abstract])**.

Change in 8.5.1.2 [expr.call] paragraph 4:

> ... When a function is called, ~~the parameters that have object type shall have completely-defined object type~~ **the type of a parameter shall not be an incomplete or abstract class type**. [Note: this still allows a parameter to be a pointer or reference to an incomplete class type. However, it prevents a passed-by-value parameter to have an incomplete class type. -- end note] ...

Change in 11.3.5 [dcl.fct] paragraph 12:

> Types shall not be defined in return or parameter types. The type of a parameter or the return type for a function definition shall not be an incomplete **or abstract** (possibly cv-qualified) class type in the context of the function definition unless the function is deleted (11.4.3).

Editorial note: I suggest to move the statement about function definitions to 11.4 [dcl.fct.def].)

Remove 13.4 [class.abstract] paragraph 3:

> ~~An abstract class shall not be used as a parameter type, as a function return type, or as the type of an explicit conversion. Pointers and references to an abstract class can be declared. [Example:~~
>
> ```
> shape x;                        // error: object of abstract class
> shape* p;                       // OK
> shape f();                      // error
> void g(shape);                  // error
> shape& h(shape&);               // OK
> ```
>
> ~~-- end example]~~